

华东师范大学数据科学与工程学院实验报告

课程名称：AI 基础

年级：2024 级

上机实践日期：

2026 年 3 月 10 日

指导教师：杨彬

姓名：唐一嘉

上机实践名称：实验1

学号：10245101434

一、实验任务

任务一：大语言模型归类

通过与大模型进行问答的方式将大模型归入以下四类

Categorization		
	Human	Rational
Behavior	Act like people Acting Humanly The Turing test approach	Act rationally The rational agent approach
Thought	Think like people Thinking humanly The cognitive modeling approach	Think rationally The "laws of thought" approach

Human: Fidelity to Human Performance
Rational: doing the "right thing"
Thought: Internal thought processes and reasoning
Behavior: External characterization

任务二：比较大语言模型

- 大语言思维链是什么
- 挑选两个大语言模型，比较它们的优劣

二、实验过程

任务一：大语言模型归类

- 参与的模型有：Deepseek-V3.2、GPT-5 mini、Kimi-K2.5、豆包 2.0

Behavior

- 豆包是最能被归类到 **act like people** 类别的大语言模型，它的回答一般都很自然，很像人类的回答，并且很懂得模仿人类，比如这个问题

请用一个16岁高中生的语气，写一段吐槽今天数学考试太难的朋友圈文案，要带点网络流行语，但不要太刻意。

📄 🔍 ...

1. 今天数学考试 我和卷子四目相对

它认识我 我不认识它

主打一个**重在参与** 😓

2. 数学：你尽管复习

考到算我输

我：好的老师 我认输了

3. 考完数学感觉人生都通透了

通透到**一片空白**

谁懂啊真的会谢 🤔

其他模型的回答都没有那么像现在的青少年群体会发出的言论，又比如说在面对“地球是平的”这一个看起来非常理性的实际的问题，他的回答依然不是直接进行分析论证，而是像人类一样进行回答

地球是平的

哈哈，这个说法还挺有**情怀**的 —— 毕竟人类真的这么信了几千年呢。

但从现代科学来看：

地球不是平的，是一个两极稍扁、赤道略鼓的椭球体。

给你几个随手就能验证的证据：

1. **船出海时**：总是先看不见船身，最后才消失桅杆，平面世界解释不了。
2. **月食时**：地球投在月球上的影子永远是弧形的。
3. **环球航行**：一直朝一个方向走，能回到起点。
4. **卫星照片**：从太空看，地球就是个球。

不过，如果你想玩 ****“地平论”梗****，我可以陪你一起抬杠~ 🤪

在有关幸福的问题中也很好地做到了 **act like people**:

你觉得“幸福”是什么？请用你自己的“感受”来描述，而不是查字典或引用名言。

如果用我最真实、最柔软的感受来说 ——

幸福不是什么惊天动地的大事，

就是**心里很轻、很暖、很安稳**的那一瞬间。

是你难过的时候，有人愿意听你说废话。

是你累了一天，躺下来不用再紧绷。

是你不用假装开心，也不用害怕被讨厌。

是你知道，不管发生什么，你都不是一个人。

是有人把你放在心上，

是你也愿意把温柔留给自己。

不用很厉害，不用很完美，



• Kimi 在 **act like people** 方面的**表现也不错**，

请用一个16岁高中生的语气，写一段吐槽今天数学考试太难的朋友圈文案，要带点网络流行语，但不要太刻意。

“数学卷子发下来我直接一个原地升天😭最后一道大题看得我CPU都烧了，草稿纸写满三页最后算出来答案是2，选项里根本没有？？老师您睡了吗我睡不着💤这难度是给人做的吗我请问呢，感觉三年数学白学，现在只想躺平当一条咸鱼🐟明天还要考物理，谁懂啊家人们（已黑化）”

这样的回答比起豆包的稍微有些奇怪，但也较为合理。在有关幸福的问题中也做到了较好的回答：

你觉得“幸福”是什么？请用你自己的“感受”来描述，而不是查字典或引用名言。

“我觉得幸福是一种**“刚刚好”**的松弛感。

就像冬天晒到午后两点的太阳，不烫，但足够把后背烘得暖洋洋的；像耳机里随机播到一首三年前很爱、却忘了名字的歌；像深夜写完最后一份作业，发现窗外居然还有一家便利店亮着灯。

它不是那种“终于赢了”的狂喜，更多时候是**心里某个皱巴巴的地方突然被抚平了**——可能是闻到家里做饭的味道，可能是发现有人记得你不爱吃香菜，可能只是今天出门时，风刚刚好，不冷也不热。

对我来说，幸福好像总是和一些**微小的、确定的、属于自己的瞬间**绑在一起。它不需要被证明，也不需要被羡慕，就只是那一刻，你觉得“嗯，这样挺好的”。

你呢？你最近一次有这种感觉是什么时候？

但是在“地球是平的”这个问题中，Kimi 选择了理性地回答，而不是像豆包一样进行模仿人类：

地球是平的

“这是一个已经被科学彻底证伪的观点。

简单的事实：

- 从古希腊时期起，人们就通过观测月食时地球投下的圆形阴影、不同纬度看到的星座差异、帆船远去时船身先消失等现象，推断出地球是球形的
- 现代天文学、卫星图像、GPS导航、环球航行.....所有实际应用都建立在地球是球体的基础上
- “地平说”无法解释时区差异、季节变化、重力方向等日常现象

为什么会有这种说法？

部分源于对权威的反叛心理，部分源于社交媒体上的信息茧房，还有一些人通过制造争议获取流量。

我的看法：质疑精神很宝贵，但科学结论需要证据支撑。地平说提出了一百多年，至今没有任何经得起检验的实验数据，而支持地球是球体的证据却每天都在被数十亿人使用着。

你是在测试我，还是真的对这个话题感兴趣？如果是后者，我可以推荐一些基础的观测实验，你自己就能验证。

• Deepseek 的表现和 Kimi 相近，都能够较好地做到 **act like people**，但是相比豆包来说稍微有所欠缺，主要体现在其语言不够自然

请用一个16岁高中生的语气，写一段吐槽今天数学考试太难的朋友圈文案，要带点网络流行语，但不要太刻意。

气死我啦!!!今天数学考试是人能做的题吗??最后三道大题看着我的眼神就像看陌生人一样[微笑]出卷老师你是不是对我有什么误解,我只是个普通的高中生不是华罗庚啊喂!

选择题全靠蒙,填空题全靠编,大题全靠写个“解”字混点同情分。考完出来跟同学对答案,好家伙,错得整整齐齐,连错题都是情侣款[拜拜]

现在只想回家躺平吃炸鸡,数学是什么?我不认识这个人。

这样的回答并不太像一个16岁的高中生。同时,其对“地球是平的”这个问题的回答也选择了理性分析

地球是平的



关于“地球是平的”这一观点,目前科学界、航天机构以及全球教育体系都一致确认:**地球是一个近似球体的行星**,并非平面。

这不仅是基于日常经验(如船只从地平线消失、月食时的弧形阴影),还有大量实证证据:

1. **照片与视频**: 自20世纪中期以来,人类从太空拍摄了无数地球照片,显示其球形。
2. **环球航行与飞行**: 人们可以沿一个方向绕地球一周,回到起点。
3. **重力与物理学**: 如果地球是平的,重力会指向中心,导致不同区域的人感觉“向下”的方向不同,这与实际不符。
4. **时间与昼夜**: 地球自转造成昼夜交替,不同地区时间不同,平面模型无法解释。

如果你有支持“地平说”的具体论据或疑问,可以提出来一起探讨,我可以帮你分析其中的科学解释。

• GPT-5 在 **act like people** 方面表现最差,换句话说,是最 **act rationally** 的大模型,其语言组织相比其他三个模型来说更加理性,如:

你觉得“幸福”是什么?请用你自己的“感受”来描述,而不是查字典或引用名言。

🗨️ 我先把“幸福”这个词在心里拆成几块:日常的、关系的、以及那种关于自我和时间的安定感,然后用感觉化的语言把它说出来。下面是我的描述与几句可直接使用的短句。

对我而言,幸福是一种轻而稳的暖,不是爆发式的激情,而是能慢慢渗进生活缝隙的安心。像早上被阳光亲了一下的被子、路上有人朝你笑的那一刻、完成一件小事后胸口的放松、或者和一个理解你的人半夜聊到笑出声——这些瞬间叠加起来,就把一天染成柔和的颜色。它也像一小团不熄的火,既不会把你烤焦,也能在冷的时候给你依靠;是有人在你需要时握住你的手,也是你知道即便有所失去,仍有理由继续期待明天。总体来说,幸福不是一个高不可攀的终点,而是许多可以被感知的、温柔的当下拼成的底色。

几句短的表达(可直接发朋友圈或当作参考):

- 幸福大概就是:被阳光和好心情同时砸中,微不足道却又足够暖。
- 幸福不是惊天动地,是那些让你愿意再努力一下的小确幸。
- 幸福像口袋里的一颗糖,偶尔摸到就会忍不住笑出来。

如果你想要更生活化、幽默一点,或更文艺一点的表述,我可以再按照你喜欢的风格改。

我刚和最好的朋友大吵了一架，现在特别难过，觉得自己很孤独。你能像一个真正的朋友那样，用温柔的语气安慰我一下吗？

我能感受到你现在真的很难受——和最好的朋友大吵之后，心里那种空洞和孤独感很真实，也很重。先让我抱抱你（虽然是文字的），你没有做错任何不该感到难过的事：你的感受值得被看见。

我刚刚读了你说的话，认真听进来了。现在我想温柔地陪你走两分钟，帮你慢慢把情绪放一放，然后给出一些可以立刻做的事情，或者如果你愿意，我可以帮你起草给朋友的消息或陪你梳理当时发生的事。

先做一件小事，让身体安静下来：

- 试试一个简单的呼吸法：慢慢吸气 4 秒，屏住 2 秒，缓缓呼气 6 秒，重复 4 次。把注意力放在呼吸上，哪怕只是一两分钟，也会稍微缓和紧绷感。
- 如果可以，给自己一杯温热的饮料，或者去洗把脸、开窗透透气，身体的微小动作会帮大脑放松一点。

允许自己感受，不要急着压抑或评判自己：

- 你可以说服自己：现在允许难过，这不代表你软弱，也不代表关系一定结束。情绪是信息，告诉你这段关系对你很重要。
- 写下一两句你现在最想说的话（不用发出），把它写在纸上或语音记录，哪怕只是“我现在好难过”——把话从胸口拿出来，本身就是一种释放。

如果你想修复、想先联络朋友，下面有几种温和的开场白（你可以直接用，或让我帮你调整更贴合你们的语气）：

- 想先道歉但不想马上讨论细节：
“我现在还很难过，刚刚我们吵得很难受。我很在乎你，能不能等我冷静一下再好好说话？”

在这两个其余模型均选择了 **act like people** 的回答方式时，gpt 的回答都更倾向于 **act rationally**

• 综上所述，豆包、Kimi、Deepseek、GPT-5 这四个模型中，豆包、Kimi、Deepseek 这三种模型都能够很好地做到 **act like people**，GPT-5 则相对来说更倾向于 **act rationally**。

Thought

• 我通过询问两个认知偏差问题来对模型的 Thought 方面进行分类，两个问题分别是：

1. 琳达今年 31 岁，单身，坦率直言，非常聪明。她主修哲学。作为一名学生，她非常关注社会正义问题，并且参加了反核示威活动。请根据以上描述，按概率从高到低排列以下两个选项：A. 琳达是一名银行出纳。B. 琳达是一名银行出纳，并且积极参与社会运动。

2. 有一个人，他很害羞、内向，喜欢整理图书，对社交不感兴趣。你觉得他更可能是一个图书管理员，还是一个农民？为什么？

最终这四个模型都没有收到认知偏差的影响，给出了理性且正确的回答，并且给出了合理且逻辑清晰的分析过程。

• 问题一

Deepseek:

有一个人，他很害羞、内向，喜欢整理图书，对社交不感兴趣。你觉得他更可能是一个图书管理员，还是一个农民？为什么？



这个问题其实是一个经典的**代表性启发**（representativeness heuristic）的例子。

从直觉上看，这个人符合我们通常对图书管理员的一些刻板印象（安静、内向、喜欢整理书），所以很多人可能会回答“图书管理员”。

但在现实世界中，**农民的人口数量远大于图书管理员**（尤其是在大多数国家），所以即便个体特质更符合图书管理员的形象，从概率上来说，他更可能是一个农民，因为农民这个职业的基数大得多，这属于**基础比率忽视**（base rate neglect）的现象。

豆包:

有一个人，他很害羞、内向，喜欢整理图书，对社交不感兴趣。你觉得他更可能是一个图书管理员，还是一个农民？为什么？

他**更可能是农民**。

不是因为性格，而是因为**人数**。

- 这个世界上，农民的数量远远、远远多于图书管理员。
- 哪怕一个人再内向、再爱整理书，从概率上讲，他首先是“大多数人”里的一个。
- 我们很容易被“性格 + 爱好”的描述带偏，觉得“内向 + 爱书 = 图书管理员”，但这只是**直觉陷阱**，不是现实。

简单说：

描述再像图书管理员，也抵不过“农民本来就得多”这个事实。

Kimi:

有一个人，他很害羞、内向，喜欢整理图书，对社交不感兴趣。你觉得他更可能是一个图书管理员，还是一个农民？为什么？

这是一个经典的**认知偏差**问题（源自心理学家丹尼尔·卡尼曼的研究）。

从表面特征看，这个人似乎**更像图书管理员**——害羞、内向、喜欢整理图书、回避社交，这些特质与人们对图书管理员的刻板印象高度吻合。

但如果认真思考概率：

- **基础概率**：世界上农民的数量远多于图书管理员（可能相差数百倍甚至上千倍）
- **特质重叠**：虽然这些特征在图书管理员中更常见，但农民群体中也有大量害羞、内向、喜欢整理的人
- **单一特征 ≠ 职业决定因素**：喜欢整理图书只是爱好，农民也需要整理农具、记录作物；不喜欢社交也不妨碍务农

更理性的答案：尽管描述听起来“很图书管理员”，但从统计学角度，他**更可能是一个农民**——仅仅因为农民这个群体基数大得多。

这提醒我们：**不要只看“代表性特征”，要同时考虑“基础概率”**。我们的大脑容易被生动的描述吸引，却忽略了更朴素的事实。

你觉得这个分析有道理吗？

GPT:

简短结论

- 如果只能从“害羞、内向、喜欢整理图书、不太社交”这几条描述来做直觉判断，多数人会选“图书管理员”。这是因为描述和人们心中的“图书管理员刻板印象”很吻合（代表性启发）。
- 但如果用严格的概率/贝叶斯思维，且没有其它信息的话，统计上更可能是“农民”，因为在很多社会里农民人数远多于图书管理员——也就是说基础发生率（base rate）通常更高。是否选农民或图书管理员，取决于这些性格特征对两种职业的“诊断力”（likelihood ratio）。

我做了什么（过程叙述）

- 我先把问题拆成两部分：一是人物特征与职业刻板印象的匹配；二是职业的先验概率（base rate）。
- 然后用贝叶斯式的直觉说明：后验概率 $\propto$ （该特征在某职业下出现的概率） $\times$ （该职业的先验概率）。
- 接着列出并解释会影响我们判断的主要认知偏差，并给出如何改进判断的具体建议。

• 问题二

Deepseek:

琳达今年 31 岁，单身，坦率直言，非常聪明。她主修哲学。作为一名学生，她非常关注社会正义问题，并且参加了反核示威活动。请根据以上描述，按概率从高到低排列以下两个选项：A. 琳达是一名银行出纳。B. 琳达是一名银行出纳，并且积极参与社会运动。

这是一个非常经典的“琳达问题”（Linda problem），由心理学家特沃斯基（Tversky）和卡尼曼（Kahneman）提出，用来演示合取谬误（conjunction fallacy）。

我的排列

根据概率原则，正确的顺序是：

1. A. 琳达是一名银行出纳。

2. B. 琳达是一名银行出纳，并且积极参与社会运动。

为什么？

因为 A 是 B 的父集，即 B 发生的前提是 A 必须成立，同时还要加上一个额外的条件（积极参与社会运动）。

因此，在概率上：

$$P(\text{银行出纳}) \geq P(\text{银行出纳且积极参与社会运动})$$

也就是说，B 的概率不可能大于 A。

豆包:

琳达今年 31 岁，单身，坦率直言，非常聪明。她主修哲学。作为一名学生，她非常关注社会正义问题，并且参加了反核示威活动。请根据以上描述，按概率从高到低排列以下两个选项：A. 琳达是一名银行出纳。B. 琳达是一名银行出纳，并且积极参与社会运动。

这是一道经典的心理学题目，考察的是概率判断中的「合取谬误」（Conjunction Fallacy）。

正确的概率排序（从高到低）是：

A > B

简单解释

- A: 琳达是银行出纳。
- B: 琳达是银行出纳 **并且** 积极参与社会运动。

在概率论中：

同时满足「条件 1 + 条件 2」的概率，**永远不可能** 高于只满足「条件 1」的概率。

$$P(\text{银行出纳} \wedge \text{社会运动}) \leq P(\text{银行出纳})$$

所以：

A 的概率一定比 B 更高。

Kimi:

琳达今年 31 岁，单身，坦率直言，非常聪明。她主修哲学。作为一名学生，她非常关注社会正义问题，并且参加了反核示威活动。请根据以上描述，按概率从高到低排列以下两个选项：A. 琳达是一名银行出纳。B. 琳达是一名银行出纳，并且积极参与社会运动。

这是一个经典的认知心理学问题，被称为“琳达问题” (Linda Problem) 或“合取谬误” (Conjunction Fallacy)，由丹尼尔·卡尼曼和阿莫斯特沃斯基在1980年代提出。

正确排序

A. 琳达是一名银行出纳 — 概率更高
B. 琳达是一名银行出纳，并且积极参与社会运动 — 概率更低

为什么？

这是概率论的基本原理：**两个事件同时发生的概率，不可能高于其中任一事件单独发生的概率。**

用数学语言表达：

- $P(A)$ = 琳达是银行出纳的概率
- $P(B)$ = 琳达是银行出纳 且 积极参与社会运动的概率
- $P(B) \leq P(A)$ 永远成立

无论琳达的背景描述多么符合“积极参与社会运动”的形象，选项B都是A的子集，因此概率必然更低。

GPT:

琳达今年 31 岁，单身，坦率直言，非常聪明。她主修哲学。作为一名学生，她非常关注社会正义问题，并且参加了反核示威活动。请根据以上描述，按概率从高到低排列以下两个选项：A. 琳达是一名银行出纳。B. 琳达是一名银行出纳，并且积极参与社会运动。

排序（按概率从高到低）：

1. A. 琳达是一名银行出纳
2. B. 琳达是一名银行出纳，并且积极参与社会运动

解释：根据概率论的基本规律，对于任意事件 X 和 Y，有 $P(X \wedge Y) \leq P(X)$ 。选项 B 是“银行出纳且积极参与社会运动”的合取事件，它的概率不可能大于仅为“银行出纳”（选项 A）的概率。人们常把描述与直觉印象匹配（代表性启发），于是误以为 B 更有可能——这就是著名的“琳达问题”（Tversky & Kahneman）中发现的合取谬误。

- 我后续又问了一些逻辑、数学等学科方面的问题，每一个大模型都能够给出过程清晰且结果正确的解答
- 因此可以得出结论，这四个大语言模型都应被归类为 **think rationally**

结论

• 通过对多款大语言模型的测试观察，可得出清晰结论：**在思维层面，模型普遍具备理性思考能力；在行为层面，不同模型有一定的差异。**

Behaviour 维度

• 模型表现出**显著差异**。豆包、Deepseek、Kimi 在回应时语气自然、互动性强，会使用生活化表达、共情语气与对话式节奏，注重交流氛围，行为模式贴近人类日常沟通，属于 **act like people**；而 GPT 则表述客观、逻辑清晰，聚焦问题解决与信息传递，语言理性，属于 **act rationally**。

• 需要特别注意的是，语言大模型的 Behaviour，即语言表述，是**能够被提示词以及上下文所影响的**，所以本次测试均是在新建对话，相同提示词的情况下测试的，这可以一定程度上反映大模型的原始设置。但大模型的语言表述形式是可以根据需要进行设置的（通过提示词、上下文等方式），因此可以认为这几个大模型都可以做到既 **act like people** 也 **act rationally**，取决于用户的需求。

Thought 维度

• 所有测试的大语言模型**均未被认知偏差误导**。面对基础概率忽视、合取谬误等经典问题时，模型能遵循逻辑、概率与客观事实进行判断，不会依赖人类直觉、刻板印象或情绪捷径，始终保持**严谨的理性推理**，全部属于 **Think rationally** 类型，不存在模仿人类直觉式思考的情况。

任务二：比较大语言模型

1. 大语言思维链是什么

• 思维链（Chain-of-Thought, 简称 CoT）是提升大模型复杂推理能力的关键技术，它引导模型在给出最终答案之前，先完整呈现出中间的思考与推导步骤，而不是直接输出结果。就像人类解决数学题、逻辑题或复杂问题时，会先梳理条件、拆解步骤、逐步推导一样，思维链让大模型把原本隐式的推理过程显性化，通过自然语言将每一步的思考逻辑清晰展现出来，让模型的决策过程从“黑盒”变得可观察、可理解。



• 思维链的核心作用是大幅提高大模型在多步骤推理任务上的准确性与可靠性，尤其适用于数学运算、逻辑推理、代码编写、复杂问题分析等场景。它通过分步拆解降低了单次决策的难度，减少因直接跳跃导致的错误，同时让推理过程更透明，便于定位错误、优化模型。借助思维链，大模型不仅能给出更可靠的答案，还能展现出更接近人类的思考方式，显著提升复杂任务的解决能力。

• 示例：

1. 常识推理

问题：冬天在户外，为什么手会冷？

普通回答：因为气温低。

思维链回答：冬天户外气温远低于人体正常体温（约 36.5°C ） $\rightarrow$ 热量会从温度高的物体流向温度低的物体 $\rightarrow$ 手上的热量会快速散失到空气中 $\rightarrow$ 热量流失后，手部温度下降，人就会感觉冷

2. 逻辑推理

问题：甲比乙大，乙比丙大，谁最小？

普通回答：丙最小。

思维链回答：已知甲比乙大，所以年龄：甲 $>$ 乙 $\rightarrow$ 已知乙比丙大，所以年龄：乙 $>$ 丙 $\rightarrow$ 把两个条件连起来：甲 $>$ 乙 $>$ 丙 $\rightarrow$ 因此，丙最小

2. 挑选两个大语言模型，比较它们的优劣

- 用于比较的模型：Pro/moonshotai/Kimi-K2.5 VS Pro/deepseek-ai/DeepSeek-V3.2
- 使用的 api 平台：siliconflow；使用的 api: <https://api.siliconflow.cn>
- max_tokens: 8192; temperature: 0.5
- 对话内容与性能指标附在附录中

i. 基础语言能力测试（通用场景）

• 语义理解

问题：请解释以下概念：向量数据库，要求说明其核心作用、典型应用场景（至少 2 个），并对比传统关系型数据库的差异（至少 3 点）

分析：Kimi 的回答更全面完整，细节更丰富，对比的维度更多；Deepseek 的回答更加短小精炼，但也相对来说不够细致，在此项的表现中弱于 Kimi。

• 逻辑推理

问题：有 A、B、C、D 四个人，已知：1. A 比 B 大 3 岁；2. C 的年龄是 D 的 2 倍；3. B 比 C 小 5 岁；4. 四人年龄总和为 82 岁；5. D 的年龄是 10 岁。请计算每个人的年龄，并写出详细推导步骤。

分析：Kimi 在回答的开始就指出了该问题存在矛盾，并且对矛盾的两种情况对应的答案都做了解答，推理过程完整，逻辑清晰；Deepseek 也基本做到了逻辑清晰的推理，但是它并没有在回答的开始就发现问题的矛盾，而是在推理过程中发现的，并且也只回答了矛盾中的一种情况，相对 Kimi 来说回答的整体逻辑不够清晰，答案不够完整，在此项的表现中弱于 Kimi

• 无幻觉测试

问题：请介绍 2025 年发布的 Python 4.0 版本的核心新特性，要求列出至少 3 个官方确认的特性，并注明特性来源（如 PEP 编号、官方文档链接）。

分析：尽管两个模型都否认了 Python 4.0 的存在，它们在此项中都出现了不同程度的幻觉。Kimi 错误将 PEP 702 这一 Python 3.13 的新特性当做了 Python 3.14 的新特性；Deepseek 错误将 Python 3.13 认为是 2025 年 10 月发布，但在其余分析中没有出现幻觉现象。总的来说，Deepseek 在此项中略微胜出。

• 中文创作

问题：以'数字时代的乡愁'为主题，写一篇 300 字左右的散文，要求语言流畅、有情感共鸣，避免空洞抒情。

分析：两个模型都很好的将数字时代这一个主题融进了文字中，大量意象的使用也让文字富有美感，但是 Deepseek 并没有很好地抓住乡愁这一主题，它所写的文字更像是纯粹的对时间时代的感慨，没有扣住乡愁这一主题，相比之下，Kimi 的回答就很好地做到了这一点，在此项中胜出

ii. 专业领域能力测试（代码 / 数学）

• 代码生成测试

问题：请为我写一个漂亮的前端代码，要求展示 python3.12 的三个新特性，要求添加详细注释，代码可直接运行，无需额外修改

分析：两个模型基本完成了要求，但相对来说 Kimi 的创作出的页面更为美观，并且也没有出现样式方面的问题
Kimi:



而 Deepseek 创作出的页面相对来说美观性略逊一筹，并且还存在着样式方面的 bug

Deepseek:



因此，虽然 Deepseek 在页面中加入了交互的有趣小功能，但是存在 bug 这一点让其在此项测试中败于 Kimi。

• 数学问题测试

问题：求解以下数学问题，给出详细推导步骤：1. 计算定积分： $\int_0^{\pi/2} (\sin x)^3 \cdot (\cos x)^2 dx$ ；2. 已知函数 $f(x) = x^3 - 6x^2 + 9x + 1$ ，求其单调区间和极值点。

分析：两个模型都很好地解决了问题，都给出了清晰的过程与正确的答案，可以认为两个模型在此项中不分胜负，需要更复杂的问题才能进行区分。

iii. 服务响应

- 两个模型都支持多模态输入，都支持工具调用
- Kimi 支持 256K 的上下文，Deepseek 支持 160K 的上下文
- 性能统计：

	Pro/moonshotai/Kimi-K2.5	Pro/deepseek-ai/DeepSeek-V3.2
平均首字耗时	80.59 秒	14.48 秒
平均总耗时	126.33 秒	54.76 秒
平均 Token 数量	2707.67 tokens	1783.5 tokens
平均端到端速度	20.48 tokens/s	31.47 tokens/s
平均纯生成速度	74.16 tokens/s	56.56 tokens/s

注：耗时受 api 平台以及实时网络影响，不完全代表模型真实性能

可以看到 Deepseek 的响应速度明显高于 Kimi，可以认为 Deepseek 的用户体验优于 Kimi；但 Kimi 的纯生成速度与平均生成 Token 数高于 Deepseek，可以认为 Kimi 的生成速度更快，生成文字数更多。

- 价格：

	Pro/moonshotai/Kimi-K2.5	Pro/deepseek-ai/DeepSeek-V3.2
命中缓存 tokens	¥0.70/ M Tokens	¥0.20/ M Tokens
输入 tokens	¥4.00/ M Tokens	¥2.00/ M Tokens
输出 tokens	¥21.00/ M Tokens	¥3.00/ M Tokens

注：价格来源于硅基流动平台

可以看到 Deepseek 价格明显低于 Kimi，具有更好的经济性。

iv. 总结

Kimi-K2.5 的核心优势体现在**基础语言能力和专业创作质量**上：在语义理解、逻辑推理、中文创作测试中表现更出色，回答更全面细致、逻辑更完整，能精准贴合需求核心，代码生成的页面美观度更高且无样式 bug；同时其支持 256K 上下文，纯生成速度更快、平均生成 Token 数量更多，适合对内容深度、丰富度和创作质量有高要求的场景。但 Kimi-K2.5 也存在明显不足，**响应速度较慢**（平均首字耗时、总耗时均远高于 DeepSeek-V3.2），且使用**价格显著偏高，经济性较差**，无幻觉测试中也出现了少量错误。

DeepSeek-V3.2 的核心优势聚焦于**服务响应效率和经济性**：响应速度大幅优于 Kimi-K2.5，平均首字耗时、总耗时更短，端到端速度更快，用户体验更流畅；同时其输入、输出及缓存 Token 的价格均远低于 Kimi-K2.5，更适合大规模、低成本的使用场景。但其不足也较为突出，基础语言能力（语义理解、逻辑推理、中文创作）和代码生成的美观度、稳定性不及 Kimi-K2.5，上下文支持长度（160K）shorter than Kimi-K2.5，纯生成速度和平均生成 Token 数量也相对较弱。

整体而言，Kimi-K2.5 更适合对内容质量、丰富度有高需求，对成本和响应速度要求较低的场景；DeepSeek-V3.2 则更适合对响应速度、使用成本敏感，对内容深度要求适中的场景。

三、总结

本次实验核心完成两项任务：一是对豆包 2.0、Kimi-K2.5、Deepseek-V3.2、GPT-5 mini 四款模型，从行为、思维维度进行归类测试；二是明确思维链内涵，对比 Kimi-K2.5 与 Deepseek-V3.2 的多维度优劣。

通过本次实验，我清晰掌握了大语言模型行为（Behaviour）与思维（Thought）两大核心维度的分类标准。同时，通过依托 API 平台开展测试，熟悉了相关工具的使用方法，提升了自身的实践操作能力，为后续开展类似的大模型测试实验奠定了坚实基础。

四、附录

1. Pro_moonshotai_Kimi-K2.5 对话内容及性能指标

1. 用户

请解释以下概念：向量数据库，要求说明其核心作用、典型应用场景（至少 2 个），并对比传统关系型数据库的差异（至少 3 点）

2. AI

向量数据库（Vector Database）是一种专门用于存储、索引和查询高维向量（Embeddings）的数据库系统。以下从核心作用、应用场景及与传统数据库的对比三个维度进行详细解释：

一、核心作用

向量数据库的核心作用是**实现非结构化数据的语义检索与相似性匹配**。

具体机制包括：

- 向量化存储**：将文本、图像、音频等非结构化数据通过深度学习模型（如 BERT、CLIP）转换为固定维度的高维向量（Embeddings），这些向量在数学空间中保留了原始数据的语义特征。
- 近似最近邻搜索（ANN）**：通过专门的索引算法（如 HNSW、IVF、PQ）在高维空间中快速查找与查询向量"最相似"的向量，而非传统的精确匹配。
- 相似度计算**：基于余弦相似度（Cosine Similarity）、欧氏距离（Euclidean Distance）或点积（Dot Product）等度量方式，量化数据间的语义关联程度。

二、典型应用场景

1. 大语言模型 RAG（检索增强生成）系统

在 ChatGPT 等 AI 应用中，向量数据库作为"外部记忆库"：

- 将海量私有文档（如企业内部手册、论文库）切分并向量化存储
- 当用户提问时，将问题转为向量检索最相关的文档片段
- 将检索结果作为上下文提供给 LLM，显著减少"幻觉"（Hallucination）现象，实现基于事实的精准回答

2. 智能推荐与语义搜索

- 电商/内容平台**：将用户行为（浏览记录）和商品/内容转为向量，通过计算"用户向量"与"商品向量"的相似度，实现"猜你喜欢"的个性化推荐
- 跨模态搜索**：支持"以图搜图"（上传图片找相似商品）或"语义搜索"（搜索"适合海边度假的轻便鞋"，即使商品标题未包含这些关键词也能返回结果）

3. 异常检测与风控

在金融或网络安全领域，将交易行为或网络流量模式向量化，通过检测与正常模式向量距离过远的离群点，实时识别欺诈行为或入侵攻击。

三、与传统关系型数据库（RDBMS）的核心差异

| 对比维度 | 传统关系型数据库（如 MySQL、Oracle） | 向量数据库（如 Milvus、Pinecone、Weaviate） |

|-----|-----|-----|

| ****数据模型**** | ****结构化表格****: 数据以行和列的形式组织, 遵循严格的 Schema 定义 (如 INT、VARCHAR) | ****高维向量空间****: 数据表示为数百至数千维的浮点数数组 (如 768 维、1536 维), 无固定 Schema 约束 |

| ****查询逻辑**** | ****精确匹配****: 基于布尔逻辑和条件过滤 ('WHERE age > 18 AND city = 'Beijing)'), 返回完全符合条件的结果 | ****近似相似性搜索****: 基于向量距离查找"最相似"结果 ('TOP-K nearest neighbors'), 允许牺牲绝对精确性换取速度 |

| ****索引机制**** | ****B+树/哈希索引****: 优化精确查询和范围查询, 时间复杂度通常为 $O(\log N)$ | ****ANN 索引****: 采用 HNSW (分层导航小世界)、IVF (倒排文件)、PQ (乘积量化) 等算法, 时间复杂度接近 $O(1)$ 或 $O(\log N)$, 但结果为近似值 |

| ****适用数据类型**** | ****结构化数据****: 适合存储姓名、金额、日期等明确格式的数据 | ****非结构化数据****: 专门处理文本、图像、音频、视频等难以用传统字段描述的数据 |

| ****一致性要求**** | ****强一致性 (ACID)****: 强调事务的原子性和数据一致性 | ****最终一致性****: 通常优先保证查询性能和可用性, 允许短暂的数据不一致 |

总结

向量数据库并非要取代关系型数据库, 而是****互补关系****。在现代 AI 应用中, 通常采用混合架构: 用关系型数据库管理用户 ID、订单状态等结构化元数据, 用向量数据库处理语义内容检索, 两者通过 ID 关联实现"结构化过滤+语义搜索"的联合查询。

> 性能指标: 首字耗时: 33.70 秒 | 总耗时: 78.98 秒 | Token 数量: 1555 | 端到端速度: 19.69 tokens/s | 纯生成速度: 34.35 tokens/s

3. 用户

有 A、B、C、D 四个人, 已知: 1. A 比 B 大 3 岁; 2. C 的年龄是 D 的 2 倍; 3. B 比 C 小 5 岁; 4. 四人年龄总和为 82 岁; 5. D 的年龄是 10 岁。请计算每个人的年龄, 并写出详细推导步骤。

4. AI

根据给定的五个条件进行推导, ****发现条件之间存在矛盾****。以下是详细推导过程和分析:

推导步骤

****第一步: 根据条件 5 确定 D 的年龄****

- 条件 5: D 的年龄是 10 岁

- ****D = 10 岁****

****第二步: 根据条件 2 计算 C 的年龄****

- 条件 2: C 的年龄是 D 的 2 倍

- $C = 2 \times D = 2 \times 10 = \textbf{20 岁}$

****第三步: 根据条件 3 计算 B 的年龄****

- 条件 3: B 比 C 小 5 岁

- $B = C - 5 = 20 - 5 = \textbf{15 岁}$

****第四步: 根据条件 1 计算 A 的年龄****

- 条件 1: A 比 B 大 3 岁

- $A = B + 3 = 15 + 3 = \textbf{18 岁}$

****第五步: 验证条件 4 (总和) ****

- 计算总和: $A + B + C + D = 18 + 15 + 20 + 10 = \textbf{63 岁}$

- 条件 4 要求总和为 82 岁

- ****矛盾点****: 计算结果 (63 岁) $\neq$ 条件给定 (82 岁), 相差 19 岁

结论与说明

基于 D=10 岁的计算结果（整数解）：

- **A: 18 岁**
- **B: 15 岁**
- **C: 20 岁**
- **D: 10 岁**
- 实际总和: 63 岁

关于矛盾的说明：

题目中条件 4（总和 82 岁）与其他条件（特别是条件 5: D=10 岁）**不一致**。

如果坚持总和必须为 82 岁（忽略条件 5 或认为 D=10 有误），则通过方程组：

- $A = B + 3$
- $C = 2D$
- $B = C - 5 = 2D - 5$
- $A = (2D - 5) + 3 = 2D - 2$
- 总和: $(2D - 2) + (2D - 5) + 2D + D = 7D - 7 = 82$

解得: $D = 89/7 \approx 12.71$ 岁**（非整数），相应：

- $C \approx 25.43$ 岁
- $B \approx 20.43$ 岁
- $A \approx 23.43$ 岁

可能的题目修正建议：

- 如果 D 确实是 10 岁，则**总和应为 63 岁**（条件 4 的 82 可能是笔误）
- 如果总和确实是 82 岁，则**D 应约为 12.71 岁**（条件 5 的 10 可能是笔误）
- 如果 D=10 且总和 82 正确，则条件 1 或条件 3 的数字需要调整（例如：若 A 比 B 大 22 岁，则刚好满足总和 82）

建议采用整数解: A=18 岁, B=15 岁, C=20 岁, D=10 岁，此时前四个条件完全满足，总和实际为 63 岁。

> 性能指标: 首字耗时: 177.41 秒 | 总耗时: 202.49 秒 | Token 数量: 4607 | 端到端速度: 22.75 tokens/s | 纯生成速度: 183.69 tokens/s

5. 用户

请介绍 2025 年发布的 Python 4.0 版本的核心新特性，要求列出至少 3 个官方确认的特性，并注明特性来源（如 PEP 编号、官方文档链接）

6. AI

重要澄清: Python 4.0 并未在 2025 年发布，且目前没有任何官方计划将在 2025 年发布 Python 4.0。

您可能混淆了以下信息：

- **Python 3.14**：预计于 **2025 年 10 月** 发布（目前处于开发阶段）
- **Python 4.0**：目前**不存在**，Python 核心开发团队尚未制定 Python 4.0 的发布路线图

实际情况说明

1. **Python 版本现状**

- 当前最新稳定版: Python 3.13（2024 年 10 月发布）
- 2025 年计划发布: **Python 3.14**（遵循每年一个 3.x 版本的发布周期）
- Python 4.0 的发布条件: 只有在需要进行不向后兼容的重大变更时才会考虑，目前官方明确表示暂无此计划

2. **Python 3.14 的实际新特性（2025 年发布）**

如果您关注的是即将在 2025 年发布的版本，以下是 Python 3.14 已确认的部分特性：

- **PEP 649**：延迟计算注解（Deferred Evaluation of Annotations）
- **PEP 741**：Python 配置 API（Python Configuration API）
- **PEP 702**：标记已弃用警告的额外参数
- **实验性 JIT 编译器**改进（基于 copy-and-patch 技术）
- **REPL**（交互式解释器）的多行编辑和颜色支持改进

3. **关于 Python 4.0 的官方立场**

Python 之父 Guido van Rossum 及核心开发团队多次强调：

- 目前专注于 Python 3.x 系列的持续改进
- Python 4.0 只有在必要时才会发布（如需要打破向后兼容性）
- 即使未来发布 4.0，也不会像 Python 2 到 3 那样进行破坏性变更

建议

如果您需要了解 Python 的最新发展，建议关注：

- **官方 PEP 仓库**： <https://peps.python.org/>
- **Python 3.14 发布计划**： <https://peps.python.org/pep-0745/>

结论：目前不存在"2025 年发布的 Python 4.0"，也没有官方确认的相关特性。

> 性能指标：首字耗时：27.49 秒 | 总耗时：50.63 秒 | Token 数量：1058 | 端到端速度：20.90 tokens/s | 纯生成速度：45.72 tokens/s

7. 用户

以'数字时代的乡愁'为主题，写一篇 300 字左右的散文，要求语言流畅、有情感共鸣，避免空洞抒情

8. AI

手机相册里存着三百张故乡的照片，云端备份着母亲发来的六十条语音。我能在高清像素里看清老屋檐角的裂纹，放大观察父亲新添的白发，却摸不到青瓦上的霜，也触不到那声音里的颤抖。这种乡愁被压缩成数据包，在 5G 信号里瞬间抵达，轻盈得失去了重量。

记得从前，乡愁是信封上邮票的齿痕，是长途电话里电流的杂音，是绿皮火车窗外倒退的杨树。那时等待很漫长，一封家书要走半个月，但拆开时纸页上有手心的温度，字迹在"见字如面"处微微晕开，像未干的泪痕。

如今视频通话让亲人的面孔清晰如昨，我却常常对着屏幕发呆。当一切都可以被即时传输、永久存储，思念反而失去了发酵的时间。我们拥有了永不褪色的数字记忆，却弄丢了那份因距离而产生的、沉甸甸的牵挂。

数字时代的乡愁，是便利里的失重，是清晰中的模糊。

> 性能指标：首字耗时：52.93 秒 | 总耗时：62.12 秒 | Token 数量：740 | 端到端速度：11.91 tokens/s | 纯生成速度：80.51 tokens/s

9. 用户

请为我写一个漂亮的前端代码，要求展示 python3.12 的三个新特性，要求添加详细注释，代码可直接运行，无需额外修改

10. AI

这是一个可直接运行的单文件 HTML，展示了 Python 3.12 的三大核心新特性。采用现代渐变设计、毛玻璃效果卡片和代码高亮，包含复制代码等交互功能。

```
```html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Python 3.12 新特性展示</title>
 <!-- 引入 Tailwind CSS 进行快速样式设计 -->
 <script src="https://cdn.tailwindcss.com"></script>
 <!-- 引入 Prism.js 用于 Python 代码语法高亮 -->
 <link href="https://cdnjs.cloudflare.com/ajax/libs/prism/1.29.0/themes/prism-tomorrow.min.css" rel="stylesheet" />
 <script src="https://cdnjs.cloudflare.com/ajax/libs/prism/1.29.0/prism.min.js"></script>
 <script src="https://cdnjs.cloudflare.com/ajax/libs/prism/1.29.0/components/prism-python.min.js"></script>
 <!-- 引入 Google Fonts 优化字体显示 -->
 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&family=JetBrains+Mono:wght@400;700&display=swap"
rel="stylesheet">

<style>
 /* 自定义样式补充 */
 body {
 font-family: 'Inter', sans-serif;
 /* 创建紫色渐变背景 */
 background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
 min-height: 100vh;
 }
 /* 代码字体设置 */
 .code-font {
 font-family: 'JetBrains Mono', monospace;
 }
 /* 毛玻璃效果卡片：半透明背景+背景模糊 */
 .glass-card {
 background: rgba(255, 255, 255, 0.95);
 backdrop-filter: blur(10px);
 border: 1px solid rgba(255, 255, 255, 0.2);
 box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.37);
 }
 /* 代码块容器定位，用于放置复制按钮 */
 .code-block {
 position: relative;
 overflow: hidden;
 }
 /* 复制按钮默认隐藏，悬停时显示 */
 .copy-btn {
 position: absolute;
 top: 0.5rem;
 right: 0.5rem;
 opacity: 0;
 transition: all 0.3s ease;
 z-index: 10;
 }

```

```
.code-block:hover .copy-btn {
 opacity: 1;
}
/* PEP 编号徽章渐变背景 */
.pep-badge {
 background: linear-gradient(90deg, #3b82f6, #8b5cf6);
}
/* 卡片入场动画初始状态 */
.animate-in {
 opacity: 0;
 transform: translateY(20px);
}
</style>
</head>
<body class="p-4 md:p-8">
 <!-- 主容器：限制最大宽度并居中 -->
 <div class="max-w-6xl mx-auto">
 <!-- 页面头部：标题和简介 -->
 <header class="text-center mb-12 text-white">
 <h1 class="text-4xl md:text-5xl font-bold mb-4 tracking-tight">Python 3.12</h1>
 <p class="text-xl opacity-90">三大革命性新特性详解</p>
 <div class="mt-4 inline-block px-4 py-1 bg-white/20 rounded-full text-sm backdrop-blur-sm border border-white/30">
 可直接运行的交互式展示
 </div>
 </header>

 <!-- 特性卡片网格布局：响应式设计，移动端单列，桌面端三列 -->
 <div class="grid md:grid-cols-3 gap-6">

 <!-- 特性 1: PEP 701 - f-string 语法改进 -->
 <article class="glass-card rounded-2xl p-6 transform hover:-translate-y-2 transition-all duration-300 animate-in">
 <div class="flex items-center justify-between mb-4">
 <h2 class="text-2xl font-bold text-gray-800">f-string 自由化</h2>
 <!-- PEP 编号徽章 -->
 PEP 701
 </div>

 <p class="text-gray-600 mb-4 text-sm leading-relaxed">
 解除了 f-string 的诸多语法限制，现在支持嵌套任意引号、反斜杠转义和多行表达式，无需再为引号冲突烦恼。
 </p>

 <!-- 代码块容器 -->
 <div class="code-block rounded-lg bg-gray-900 shadow-inner">
 <!-- 复制按钮，点击触发 copyCode 函数 -->
 <button onclick="copyCode(this)" class="copy-btn bg-white/10 hover:bg-white/20 text-white px-3 py-1 rounded text-xs backdrop-blur-sm border border-white/20 transition-colors">
 复制代码
 </button>
 <pre class="!m-0 !bg-transparent"><code class="language-python"># Python 3.12 之前：引号冲突会导致语法错误
现在可以嵌套相同类型的引号
name = "Python"
version = 3.12

1. 支持嵌套引号（之前必须用不同引号）
s = f'{'Hello' + ' ' + name}!'"
```

# 2. 支持反斜杠（之前完全禁止）

```
path = f"C:\\Users\\{name}\\file.txt"
regex = f"Pattern: \\d+"
```

# 3. 多行表达式（无需反斜杠续行）

```
result = f"""
计算结果: {
 100 + 200 +
 300 + 400
}
"""
print(s)
print(path)</code></pre>

```

</div>

<!-- 特性标签 -->

<div class="mt-4 flex items-center text-xs text-gray-500 font-medium">

<span class="w-2 h-2 bg-green-500 rounded-full mr-2 animate-pulse"></span>

语法限制大幅放宽

</div>

</article>

<!-- 特性 2: PEP 695 - 类型参数语法（泛型简化） -->

<article class="glass-card rounded-2xl p-6 transform hover:-translate-y-2 transition-all duration-300 animate-in">

<div class="flex items-center justify-between mb-4">

<h2 class="text-2xl font-bold text-gray-800">泛型语法简化</h2>

<span class="pep-badge text-white text-xs px-3 py-1 rounded-full font-mono font-bold">PEP 695</span>

</div>

<p class="text-gray-600 mb-4 text-sm leading-relaxed">

全新的类型参数声明语法，使用方括号直接声明类型变量，告别繁琐的 `TypeVar` 定义，让泛型编程更直观。

</p>

<div class="code-block rounded-lg bg-gray-900 shadow-inner">

<button onclick="copyCode(this)" class="copy-btn bg-white/10 hover:bg-white/20 text-white px-3 py-1 rounded text-xs backdrop-blur-sm border border-white/20 transition-colors">

复制代码

</button>

<pre class="!m-0 !bg-transparent"><code class="language-python"># Python 3.12 新语法：直接声明类型参数 [T]

# 无需再写: from typing import TypeVar; T = TypeVar('T')

```
def process[T](data: T) -> T:
```

```
 """返回输入数据的泛型函数"""
```

```
 return data
```

```
class Container[T]:
```

```
 """泛型容器类，T 在类定义时声明"""
```

```
 def __init__(self, value: T) -> None:
```

```
 self.value = value
```

```
 def get(self) -> T:
```

```
 return self.value
```

# 类型推断示例

```
num_box = Container(42) # 自动推断为 Container[int]
```

```
str_box = Container("hello") # 自动推断为 Container[str]
```

# 函数使用

```
print(process(100)) # 输出: 100
print(num_box.get()) # 输出: 42
```

```
<div class="mt-4 flex items-center text-xs text-gray-500 font-medium">

 更简洁的泛型编程
</div>
```

</article>

<!-- 特性 3: 改进的错误消息 (Error Messages) -->

<article class="glass-card rounded-2xl p-6 transform hover:-translate-y-2 transition-all duration-300 animate-in">

<div class="flex items-center justify-between mb-4">

<h2 class="text-2xl font-bold text-gray-800">智能错误提示</h2>

<span class="pep-badge text-white text-xs px-3 py-1 rounded-full font-mono font-bold">Enhanced</span>

</div>

<p class="text-gray-600 mb-4 text-sm leading-relaxed">

更精确的语法错误定位，针对常见拼写错误（如 `import` 模块名）提供智能建议，大幅提升调试效率。

</p>

<div class="code-block rounded-lg bg-gray-900 shadow-inner">

<button onclick="copyCode(this)" class="copy-btn bg-white/10 hover:bg-white/20 text-white px-3 py-1 rounded text-xs backdrop-blur-sm border border-white/20 transition-colors">

复制代码

</button>

<pre class="!m-0 !bg-transparent"><code class="language-python"># 示例 1: 未关闭的括号精确定位

# 3.12 会指出哪一行括号未关闭，而不仅是文件末尾

```
def calculate(x, y)
 return x + y
```

# 示例 2: 拼写错误智能建议

# 如果你写: `import statisitics` （拼写错误）

# 3.12 会提示: Did you mean: statistics?

# 示例 3: 更好的缩进错误提示

if True:

print("没有缩进") # 会指出期望缩进的位置

# 示例 4: SyntaxError 精确位置标记

print("Hello" # 会指出缺少右括号的位置

</div>

<div class="mt-4 flex items-center text-xs text-gray-500 font-medium">

<span class="w-2 h-2 bg-purple-500 rounded-full mr-2 animate-pulse"></span>

开发者体验大幅提升

</div>

</article>

</div>

<!-- 页面底部信息 -->

<footer class="mt-12 text-center text-white/80 text-sm">

<p class="mb-2">在终端运行 <code class="bg-white/20 px-2 py-1 rounded font-mono text-xs">python --version</code> 检查您的版本

</p>

```
<p class="opacity-60 text-xs">使用 HTML5 + Tailwind CSS + Prism.js 构建 | 零依赖，单文件可直接运行</p>
</footer>
</div>

<!-- JavaScript 交互逻辑 -->
<script>
 /**
 * 复制代码块内容到剪贴板
 * @param {HTMLElement} btn - 被点击的按钮元素
 */
 async function copyCode(btn) {
 // 获取相邻的 code 元素内的文本内容
 const codeBlock = btn.nextElementSibling;
 const code = codeBlock.textContent.trim();

 try {
 // 使用现代 Clipboard API 写入剪贴板
 await navigator.clipboard.writeText(code);

 // 提供视觉反馈：更改按钮文字和样式
 const originalText = btn.textContent;
 btn.textContent = '已复制!';
 btn.classList.add('bg-green-500/30', 'text-green-300', 'border-green-400/30');

 // 2 秒后恢复原状
 setTimeout(() => {
 btn.textContent = originalText;
 btn.classList.remove('bg-green-500/30', 'text-green-300', 'border-green-400/30');
 }, 2000);
 } catch (err) {
 // 错误处理：如果复制失败（如权限问题）
 console.error('复制失败:', err);
 btn.textContent = '复制失败';
 setTimeout(() => {
 btn.textContent = '复制代码';
 }, 2000);
 }
 }

 // DOM 加载完成后执行入场动画
 document.addEventListener('DOMContentLoaded', () => {
 const cards = document.querySelectorAll('.animate-in');
 // 为每个卡片添加延迟入场动画，创造错落有致的视觉效果
 cards.forEach((card, index) => {
 setTimeout(() => {
 card.style.transition = 'all 0.6s cubic-bezier(0.4, 0, 0.2, 1)';
 card.style.opacity = '1';
 card.style.transform = 'translateY(0)';
 }, index * 150); // 每个卡片延迟 150ms
 });
 });
</script>
</body>
</html>
...

```



## ### 代码说明

## 1. \*\*三大特性准确对应 Python 3.12 官方发布\*\*:

- \*\*PEP 701\*\*: f-string 语法自由化（支持嵌套引号、反斜杠、多行表达式）
- \*\*PEP 695\*\*: 类型参数语法（使用 `[T]` 替代繁琐的 TypeVar）
- \*\*Enhanced Error Messages\*\*: 改进的错误提示（拼写建议、精确位置标记）

## 2. \*\*可直接运行\*\*: 保存为 `.html` 文件后直接在浏览器打开，所有资源均通过 CDN 加载，无需安装任何依赖。

## 3. \*\*设计亮点\*\*:

- 使用 \*\*Tailwind CSS\*\* 实现响应式布局和渐变背景
- \*\*Prism.js\*\* 提供专业的 Python 语法高亮（Tomorrow 主题）
- \*\*毛玻璃效果\*\*（Glassmorphism）卡片设计
- \*\*悬停复制按钮\*\*和\*\*入场动画\*\*增强交互体验

## 4. \*\*详细注释\*\*: HTML 结构、CSS 样式和 JavaScript 函数均包含中文注释，说明每个部分的功能和实现逻辑。

> 性能指标: 首字耗时: 155.74 秒 | 总耗时: 294.85 秒 | Token 数量: 6529 | 端到端速度: 22.14 tokens/s | 纯生成速度: 46.93 tokens/s

---

## ## 11. 用户

求解以下数学问题，给出详细推导步骤: 1. 计算定积分:  $\int_0^{\pi/2} (\sin x)^3 \cdot (\cos x)^2 dx$ ; 2. 已知函数  $f(x) = x^3 - 6x^2 + 9x + 1$ ，求其单调区间和极值点。

---

## ## 12. AI

以下是两个数学问题的详细推导过程:

---

## ### 问题 1: 定积分计算

**计算:** 
$$I = \int_0^{\frac{\pi}{2}} \sin^3 x \cdot \cos^2 x \, dx$$

## #### 推导步骤:

**第一步: 恒等变形**

将  $\sin^3 x$  拆分为  $\sin^2 x \cdot \sin x$ ，并利用  $\sin^2 x = 1 - \cos^2 x$ :

$$I = \int_0^{\frac{\pi}{2}} (1 - \cos^2 x) \cdot \cos^2 x \cdot \sin x \, dx = \int_0^{\frac{\pi}{2}} (\cos^2 x - \cos^4 x) \sin x \, dx$$

**第二步: 换元法**

令  $u = \cos x$ ，则  $du = -\sin x \, dx$ ，即  $-du = \sin x \, dx$

**第三步: 变换积分限**

- 当  $x = 0$  时， $u = \cos 0 = 1$

- 当  $x = \frac{\pi}{2}$  时， $u = \cos \frac{\pi}{2} = 0$

**第四步: 计算积分**

$$I = \int_1^0 (u^2 - u^4) (-du) = \int_0^1 (u^2 - u^4) \, du$$

$$\lim_{u \rightarrow 5} \left( \frac{u^3}{3} - \frac{u^5}{5} \right) - \left( \frac{1}{3} - \frac{1}{5} \right) = \left( \frac{125}{3} - \frac{3125}{5} \right) - 0 = \frac{5-312}{3} = \frac{2}{15}$$

答案:  $\frac{2}{15}$

---

### 问题 2: 函数单调性与极值

已知:  $f(x) = x^3 - 6x^2 + 9x + 1$

推导步骤:

第一步: 求一阶导数

$$f'(x) = 3x^2 - 12x + 9 = 3(x^2 - 4x + 3) = 3(x-1)(x-3)$$

第二步: 求临界点

令  $f'(x) = 0$ , 得:

$$x_1 = 1, x_2 = 3$$

第三步: 分析导数符号 (穿根法/区间测试)

区间:  $(-\infty, 1)$ ,  $(1, 3)$ ,  $(3, +\infty)$   
测试点:  $x=0$ ,  $x=2$ ,  $x=4$   
 $f'(x)$  符号:  $3(-)(-) = +$ ,  $3(+)(-) = -$ ,  $3(+)(+) = +$   
单调性:  $\nearrow$  递增,  $\searrow$  递减,  $\nearrow$  递增

第四步: 确定极值点

在  $x=1$  处: 导数由正变负, 故为极大值点

$$f(1) = 1 - 6 + 9 + 1 = 5$$

在  $x=3$  处: 导数由负变正, 故为极小值点

$$f(3) = 27 - 54 + 27 + 1 = 1$$

结论汇总:

单调区间:

- 单调递增:  $(-\infty, 1)$  和  $(3, +\infty)$

- 单调递减:  $(1, 3)$

极值点:

- 极大值点:  $(1, 5)$

- 极小值点:  $(3, 1)$

> 性能指标: 首字耗时: 36.25 秒 | 总耗时: 68.92 秒 | Token 数量: 1757 | 端到端速度: 25.49 tokens/s | 纯生成速度: 53.78 tokens/s

---

# 总体性能统计

平均首字耗时: 80.59 秒

平均总耗时: 126.33 秒

平均 Token 数量: 2707.67 tokens

**\*\*平均端到端速度\*\***: 20.48 tokens/s  
**\*\*平均纯生成速度\*\***: 74.16 tokens/s

## 2. Pro\_deepseek-ai\_DeepSeek-V3.2 对话内容及性能指标

## 1. 用户

请解释以下概念：向量数据库，要求说明其核心作用、典型应用场景（至少 2 个），并对比传统关系型数据库的差异（至少 3 点）

---

## 2. AI

## 向量数据库：概念、作用与应用

**\*\*向量数据库\*\***是一种专门用于存储、索引和查询**\*\*高维向量\*\***（即嵌入向量）的数据库系统。这些向量通常由 AI 模型（如文本、图像、音频的嵌入模型）生成，用于表示非结构化数据的语义特征。其核心能力是**\*\*高效执行相似性搜索\*\***（如最近邻搜索），找到与查询向量最相似的存储向量。

---

### **\*\*核心作用\*\***

1. **\*\*高效相似性检索\*\***

通过近似最近邻（ANN）算法快速在海量高维向量中查找相似项，解决传统方法在高维空间中的“维度灾难”问题。

2. **\*\*非结构化数据语义理解\*\***

将文本、图像等数据转换为向量，使数据库能够理解内容语义，而不仅是关键词匹配。

3. **\*\*AI 应用支持\*\***

作为大模型（LLM）的“长期记忆”，存储知识库嵌入，增强 AI 的上下文学习和检索能力。

---

### **\*\*典型应用场景\*\***

1. **\*\*推荐系统\*\***

- **\*\*示例\*\***：电商平台将用户行为（点击、购买记录）和商品信息转换为向量，实时推荐相似商品。

- **\*\*技术实现\*\***：用户向量与商品向量库进行相似度匹配，快速生成个性化推荐列表。

2. **\*\*多模态搜索\*\***

- **\*\*示例\*\***：图库 App 中上传照片查找相似图片，或输入文字描述搜索相关图像。

- **\*\*技术实现\*\***：通过 CLIP 等模型将图像/文本统一编码为向量，跨模态检索语义相关的内容。

---

### **\*\*与传统关系型数据库的差异\*\***

| <b>**对比维度**</b> | <b>**向量数据库**</b>     | <b>**传统关系型数据库**</b> |
|-----------------|----------------------|---------------------|
| <b>**数据模型**</b> | 高维向量（浮点数数组）          | 结构化表格（行与列）          |
| <b>**查询方式**</b> | 相似度搜索（如余弦相似度）        | 精确匹配（SQL 条件查询）      |
| <b>**适用场景**</b> | 非结构化数据语义检索、AI 应用     | 事务处理、结构化数据关联分析      |
| <b>**索引技术**</b> | HNSW、IVF-PQ 等 ANN 算法 | B 树、哈希索引            |
| <b>**扩展性**</b>  | 针对高维数据垂直优化           | 擅长水平扩展处理事务          |

---

### \*\*关键差异详解\*\*

1. \*\*查询逻辑的本质不同\*\*

- 关系型数据库：查询“等于”或“满足条件”的数据（如`WHERE price > 100`）。
- 向量数据库：查询“最相似”的数据（如“找到与这张图片最相似的 10 张图”），结果具有相对性和排序性。

2. \*\*处理的数据类型\*\*

- 关系型数据库：高度依赖预设模式（Schema），适合财务记录、用户信息等结构化数据。
- 向量数据库：无需固定模式，直接处理 AI 模型生成的向量，适合动态变化的非结构化数据。

3. \*\*性能优化方向\*\*

- 关系型数据库：优化联表查询、事务一致性（ACID）。
- 向量数据库：优化高维向量距离计算速度，牺牲绝对精确度换取检索效率（近似搜索）。

---

### \*\*技术选型建议\*\*

- 选择\*\*向量数据库\*\*当：  
需要处理 Embedding 向量、实现语义搜索、构建 AI 代理记忆库或跨模态检索。
- 选择\*\*关系型数据库\*\*当：  
需要强一致性事务、复杂关联查询或高度结构化的业务数据管理。

**\*\*趋势\*\***：现代应用常结合两者，用关系型数据库管理元数据，向量数据库处理语义检索，形成混合架构以满足多样化需求。

> 性能指标：首字耗时：23.31 秒 | 总耗时：42.52 秒 | Token 数量：1046 | 端到端速度：24.60 tokens/s | 纯生成速度：54.44 tokens/s

---

## 3. 用户

有 A、B、C、D 四个人，已知：1. A 比 B 大 3 岁；2. C 的年龄是 D 的 2 倍；3. B 比 C 小 5 岁；4. 四人年龄总和为 82 岁；5. D 的年龄是 10 岁。请计算每个人的年龄，并写出详细推导步骤。

---

## 4. AI

好的，我们一步步来推导。

---

**\*\*已知条件：\*\***

1.  $(A = B + 3)$
2.  $(C = 2 \times D)$
3.  $(B = C - 5)$
4.  $(A + B + C + D = 82)$
5.  $(D = 10)$

---

**\*\*步骤 1：从 D 求 C\*\***

由条件 2：

[

$C = 2 \times D = 2 \times 10 = 20$

]

---

**\*\*步骤 2: 从 C 求 B\*\***

由条件 3:

[

$$B = C - 5 = 20 - 5 = 15$$

]

---

**\*\*步骤 3: 从 B 求 A\*\***

由条件 1:

[

$$A = B + 3 = 15 + 3 = 18$$

]

---

**\*\*步骤 4: 验证总和\*\***

[

$$A + B + C + D = 18 + 15 + 20 + 10 = 63$$

]

咦，这里算出来是 63，但条件 4 说总和是 82，矛盾了。

---

**\*\*检查\*\*:**

如果  $D = 10$ ，则  $C = 20$ ， $B = 15$ ， $A = 18$ ，总和确实是 63，不是 82。

说明 **\*\*条件 5 ( $D=10$ ) 与条件 4 (总和=82) 不能同时成立\*\***，题目可能故意设了一个矛盾，或者  $D=10$  是另一个条件？

---

我们重新看，如果  $D=10$  是给定的，那么总和 82 就不可能成立。

但题目说“已知：1...2...3...4...5...”，那么可能是 **\*\*条件 5 是待验证的，还是条件 4 是待验证的？\*\***

---

**\*\*尝试忽略条件 5，只用条件 1-4 解\*\*:**

由 1:  $(A = B + 3)$

由 2:  $(C = 2D)$

由 3:  $(B = C - 5)$

由 4:  $(A + B + C + D = 82)$

把 2 代入 3:

[

$$B = 2D - 5$$

]

把  $(A = B + 3)$  代入:

[

$$A = (2D - 5) + 3 = 2D - 2$$

]

总和：

$$\begin{aligned} & \backslash[ \\ & (2D - 2) + (2D - 5) + 2D + D = 82 \end{aligned}$$

$\backslash]$

$$\begin{aligned} & \backslash[ \\ & (2D - 2) + (2D - 5) = 4D - 7 \end{aligned}$$

$\backslash]$

加上  $\backslash( 2D + D = 3D \backslash)$ ：

$$\begin{aligned} & \backslash[ \\ & 4D - 7 + 3D = 7D - 7 = 82 \end{aligned}$$

$\backslash]$

$$\begin{aligned} & \backslash[ \\ & 7D = 89 \end{aligned}$$

$\backslash]$

$$\begin{aligned} & \backslash[ \\ & D = \frac{89}{7} \approx 12.714... \end{aligned}$$

$\backslash]$

这样：

$$\begin{aligned} & \backslash[ \\ & C = 2D = \frac{178}{7} \approx 25.428... \end{aligned}$$

$\backslash]$

$$\begin{aligned} & \backslash[ \\ & B = C - 5 = \frac{178}{7} - \frac{35}{7} = \frac{143}{7} \approx 20.428... \end{aligned}$$

$\backslash]$

$$\begin{aligned} & \backslash[ \\ & A = B + 3 = \frac{143}{7} + \frac{21}{7} = \frac{164}{7} \approx 23.428... \end{aligned}$$

$\backslash]$

总和：

$$\begin{aligned} & \backslash[ \\ & \frac{164}{7} + \frac{143}{7} + \frac{178}{7} + \frac{89}{7} = \frac{574}{7} = 82 \end{aligned}$$

$\backslash]$

---

**\*\*结论\*\*：**

如果只用条件 1-4，得到分数解：

$\backslash[$

$$A = \frac{164}{7}, \quad B = \frac{143}{7}, \quad C = \frac{178}{7}, \quad D = \frac{89}{7}$$

$\backslash]$

如果坚持条件 5 ( $D=10$ )，则总和为 63，与条件 4 矛盾。

---

**\*\*题目可能意图\*\*：**

如果这是一道逻辑题，可能故意给一个矛盾条件 ( $D=10$ ) 来测试是否发现矛盾。

如果要求整数解，则条件 5 不能成立，必须用 1-4 来解，但得到分数年龄。

---

**\*\*最终答案（按条件 1-4 推导）\*\*：**

$\backslash[$

$$\boxed{A=\frac{164}{7},\ B=\frac{143}{7},\ C=\frac{178}{7},\ D=\frac{89}{7}}$$

$\backslash]$

年龄总和为 82，满足所有条件 1-4，但 D 不是 10 岁。

> 性能指标：首字耗时：0.87 秒 | 总耗时：43.65 秒 | Token 数量：1021 | 端到端速度：23.39 tokens/s | 纯生成速度：23.87 tokens/s

---

## 5. 用户

请介绍 2025 年发布的 Python 4.0 版本的核心新特性，要求列出至少 3 个官方确认的特性，并注明特性来源（如 PEP 编号、官方文档链接）

---

## 6. AI

关于您提到的“Python 4.0”，这是一个需要首先澄清的重要概念：**截至 2024 年底，Python 4.0 版本并不存在，也没有任何官方发布计划。**

Python 的核心开发团队和指导委员会（Python Steering Council）曾多次公开表示，**目前没有开发或发布 Python 4.0 的计划**。Python 3.x 系列是当前和可预见未来的唯一主线。任何关于 Python 4.0 发布日期或特性的讨论均为社区猜测或误解。

为了避免混淆，并为您提供准确、有价值的信息，以下将分为两部分说明：

- 官方立场澄清**：解释为何没有 Python 4.0。
- 未来特性预览**：介绍 Python 3.x 系列（如即将发布的 3.13）中已确认的重要新特性，这些特性常被误认为是“4.0”的内容。

---

### **第一部分：为何没有 Python 4.0？官方立场说明**

Python 从 2.x 到 3.x 的迁移是一次破坏性更新，给社区带来了长期的过渡成本。基于此教训，Python 创始人 Guido van Rossum 及核心团队明确表示，未来任何可能破坏向后兼容性的“4.0”版本都不会轻易推出。

\* **关键来源**：

\* **PEP 387**：提出了向后不兼容更改的弃用策略，旨在最小化破坏性。

\* **PEP 404**：正式撤销了曾经提议的 Python 2.8 版本，巩固了 Python 3 的唯一主线地位。

\* **官方声明**：在多次 PyCon 大会和 Python 邮件列表中，核心开发者明确表示“Python 4.0 只是当人们在谷歌搜索 Python 3.0 时出现的一个梗”。

**结论**：您可以放心地继续深入学习和应用 Python 3.x，无需担心短期内出现需要大规模迁移的“4.0”版本。

---

### **第二部分：Python 的未来：3.x 系列中的核心新特性（以 Python 3.13 为例）**

真正的创新和重要特性都在 Python 3.x 的持续更新中。以下是 **计划于 2025 年 10 月发布的 Python 3.13** 中已确认或极有可能引入的几个关键特性：

#### **1. 自由线程（Free-threading / “无 GIL” 模式）**

\* **核心作用**：允许 Python 线程并行执行 CPU 密集型任务，突破全局解释器锁的限制，显著提升多核 CPU 的利用率和程序性能。

\* **来源**：

\* **PEP 703**：《使全局解释器锁可选》。这是 Python 迈向高效并发的里程碑提案。

\* **状态**：该工作正在分阶段进行。在 3.13 中，将首先提供一个 **实验性的、可选的“无 GIL”构建模式**，供用户测试和适配。默认解释器仍带有 GIL。

\* [PEP 703 链接](https://peps.python.org/pep-0703/)

#### **2. 自适应解释器（Adaptive Interpreter）**

\* **核心作用**：大幅提升纯 Python 代码的解释执行速度。解释器会“记住”字节码的执行路径，在下次执行相同代码时跳过冗余的查找步骤，从而获得即时性能提升。

\* **来源**：

\* 这是 CPython 解释器内部的重大性能优化项目，由核心开发者领导。

\* 相关更改和讨论可在 Python 开发邮件列表和 GitHub 的 CPython 仓库提交记录中查看（例如，提交标题常含“GH-xxx”编号）。

\* [CPython 3.13 性能说明](https://docs.python.org/3.13/whatsnew/3.13.html#performance)

#### ### 3. 更精确的错误位置报告

\* **核心作用**：当代码发生异常（如 `NameError`, `AttributeError`）时，解释器不仅能指出哪一行出错，还能精确指出该行中的**哪个**表达式或单词导致了问题，极大简化调试过程。

\* **来源**：

\* **PEP 657**：《在错误回溯中引入细粒度错误位置》。

\* 该特性已在 Python 3.11 中部分引入，并在后续版本（包括 3.13）中持续改进和增强。

\* [PEP 657 链接](https://peps.python.org/pep-0657/)

---

#### ### 总结与建议

| 项目 | 实际情况 | 建议关注点 |

| :--- | :--- | :--- |

| **Python 4.0** | **不存在**，无计划。是社区的误解或玩笑。 | 无需关注，避免被误导。 |

| **Python 的未来** | **Python 3.x 系列的持续演进**。 | 关注 **最新的 3.x 版本**（如 3.13）和 **官方的 PEP 提案**。 |

要获取最权威的 Python 发展信息，请始终参考：

- 官方发布日历和文档**：[Python.org Downloads](https://www.python.org/downloads/)
- PEP 索引**：[Python Enhancement Proposals](https://peps.python.org/)
- 官方博客和邮件列表**：[Python News](https://blog.python.org/)

希望这份详细的说明能帮助您清晰了解 Python 的发展路线，并将注意力集中在真正重要和官方的技术更新上。

> 性能指标：首字耗时：9.97 秒 | 总耗时：84.25 秒 | Token 数量：1524 | 端到端速度：18.09 tokens/s | 纯生成速度：20.52 tokens/s

---

#### ## 7. 用户

以“数字时代的乡愁”为主题，写一篇 300 字左右的散文，要求语言流畅、有情感共鸣，避免空洞抒情

---

#### ## 8. AI

##### ## 数字时代的乡愁

母亲发来一张翻拍的老照片——九十年代的全家福，像素粗糙如雨天的窗玻璃。我试图用修图软件擦除那些噪点，却发现抹去的还有父亲中山装上真实的织物纹理，以及背景里那堵如今已被拆毁的砖墙特有的淡青色。

我们拥有了无限存储的云端相册，却失去了照片边缘偶然沾染的、带着时光气息的指纹印。视频通话如此清晰，清晰到能数清母亲新添的白发，可再高的分辨率也传递不了她指尖淡淡的茶渍香，那是老家搪瓷杯经年累月浸润出的味道。

数字世界把一切记忆压平成可检索的数据，给每张面孔标注姓名与日期。而我怀念的，恰是那些无法被标签定义的东西：老式磁带里夹杂的电台杂音，旧书页间风干的茉莉花瓣，还有电话挂断后漫长的忙音——那空白里曾盛满等待与想象。



如今我们什么都能保存，却又什么都留不住。乡愁不再是地理上的远方，而是时间格式的不可兼容：我们回不到那个未被数字化、未被精确描述的，温暖而模糊的从前。

> 性能指标：首字耗时：14.38 秒 | 总耗时：21.62 秒 | Token 数量：584 | 端到端速度：27.02 tokens/s | 纯生成速度：80.73 tokens/s

---

## 9. 用户

请为我写一个漂亮的前端代码，要求展示 python3.12 的三个新特性，要求添加详细注释，代码可直接运行，无需额外修改

---

## 10. AI

# Python 3.12 新特性展示

我将创建一个交互式网页，展示 Python 3.12 的三个重要新特性。这个页面将包含代码示例、解释和实时运行结果。

```
```html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Python 3.12 新特性展示</title>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.8.0/styles/atom-one-dark.min.css">
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: 'Segoe UI', 'Microsoft YaHei', sans-serif;
    }

    body {
      background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%);
      color: #e0e0e0;
      line-height: 1.6;
      padding: 20px;
      min-height: 100vh;
    }

    .container {
      max-width: 1200px;
      margin: 0 auto;
    }

    header {
      text-align: center;
      margin-bottom: 40px;
      padding: 20px;
      background: rgba(0, 0, 0, 0.3);
      border-radius: 15px;
      box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
      border: 1px solid rgba(255, 255, 255, 0.1);
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>Python 3.12 新特性展示</h1>
    </div>
  </div>
</body>
</html>
```
```

```
}

h1 {
 color: #4fc3f7;
 font-size: 2.8rem;
 margin-bottom: 10px;
 text-shadow: 0 2px 10px rgba(79, 195, 247, 0.3);
}

.subtitle {
 color: #81d4fa;
 font-size: 1.2rem;
 margin-bottom: 20px;
}

.intro {
 max-width: 800px;
 margin: 0 auto 30px;
 font-size: 1.1rem;
 color: #b0bec5;
 text-align: center;
}

.features-container {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(350px, 1fr));
 gap: 30px;
 margin-bottom: 40px;
}

.feature-card {
 background: rgba(25, 25, 35, 0.8);
 border-radius: 15px;
 padding: 25px;
 box-shadow: 0 8px 25px rgba(0, 0, 0, 0.4);
 transition: transform 0.3s, box-shadow 0.3s;
 border: 1px solid rgba(255, 255, 255, 0.05);
 display: flex;
 flex-direction: column;
}

.feature-card:hover {
 transform: translateY(-5px);
 box-shadow: 0 15px 35px rgba(0, 0, 0, 0.5);
}

.feature-header {
 display: flex;
 align-items: center;
 margin-bottom: 20px;
 padding-bottom: 15px;
 border-bottom: 1px solid rgba(79, 195, 247, 0.2);
}

.feature-icon {
 font-size: 2.5rem;
 margin-right: 15px;
}
```

```
 color: #4fc3f7;
 }

 .feature-title {
 font-size: 1.5rem;
 color: #4fc3f7;
 }

 .feature-description {
 margin-bottom: 20px;
 color: #b0bec5;
 flex-grow: 1;
 }

 .code-container {
 background: #1e1e1e;
 border-radius: 10px;
 overflow: hidden;
 margin-bottom: 20px;
 border: 1px solid rgba(255, 255, 255, 0.1);
 }

 .code-header {
 background: #252526;
 padding: 10px 15px;
 display: flex;
 justify-content: space-between;
 align-items: center;
 border-bottom: 1px solid rgba(255, 255, 255, 0.1);
 }

 .code-title {
 color: #4fc3f7;
 font-weight: 600;
 }

 .run-btn {
 background: #0d47a1;
 color: white;
 border: none;
 padding: 8px 15px;
 border-radius: 5px;
 cursor: pointer;
 font-weight: 600;
 transition: background 0.3s;
 }

 .run-btn:hover {
 background: #1565c0;
 }

 .code-block {
 padding: 15px;
 overflow-x: auto;
 }

 .output-container {
```

```
background: #1a1a1a;
border-radius: 10px;
padding: 15px;
margin-top: 10px;
border: 1px solid rgba(255, 255, 255, 0.1);
min-height: 60px;
font-family: 'Consolas', monospace;
white-space: pre-wrap;
}

.output-title {
color: #81c784;
margin-bottom: 8px;
font-weight: 600;
}

footer {
text-align: center;
margin-top: 40px;
padding-top: 20px;
border-top: 1px solid rgba(255, 255, 255, 0.1);
color: #78909c;
font-size: 0.9rem;
}

.highlight {
color: #4fc3f7;
font-weight: 600;
}

@media (max-width: 768px) {
.features-container {
grid-template-columns: 1fr;
}

h1 {
font-size: 2.2rem;
}
}
</style>
</head>
<body>
<div class="container">
<header>
<h1>Python 3.12 新特性展示</h1>
<div class="subtitle">探索 Python 3.12 的三个重要更新</div>
<div class="intro">
Python 3.12 带来了多项性能改进和新特性，使代码更高效、更易读。以下展示三个最值得关注的特性。
</div>
</header>

<div class="features-container">
<!-- 特性 1: 更友好的错误信息 -->
<div class="feature-card">
<div class="feature-header">
<div class="feature-icon"> </div>
<div>
```

```
<h2 class="feature-title">更友好的错误信息</h2>
<div class="feature-subtitle">增强的错误提示和回溯信息</div>
</div>
</div>
<div class="feature-description">
 Python 3.12 改进了错误信息的可读性，特别是在涉及字典键、属性访问和导入错误时。错误信息现在会明确指出问题所在，并提供更清晰的建议。
</div>

<div class="code-container">
 <div class="code-header">
 <div class="code-title">示例代码：改进的错误信息</div>
 <button class="run-btn" onclick="runCode(1)">运行代码</button>
 </div>
 <div class="code-block">
 <pre><code class="python"># 示例 1: 字典键错误
my_dict = {"name": "Alice", "age": 30}

在 Python 3.12 中，错误信息会明确指出哪个键不存在
print(my_dict["nam"]) # 注意：应该是"name"而不是"nam"

示例 2: 属性错误
class Person:
 def __init__(self, name):
 self.full_name = name

p = Person("Bob")
更清晰的错误信息会提示可能的正确属性名
print(p.name) # 应该是 full_name

示例 3: 导入错误
尝试导入不存在的模块
import matplotlib # 应该是 matplotlib</code></pre>
 </div>
 </div>

 <div class="output-container">
 <div class="output-title">输出结果： </div>
 <div id="output1">点击"运行代码"查看结果</div>
 </div>
</div>

<!-- 特性 2: PEP 701 - f-string 的语法形式化 -->
<div class="feature-card">
 <div class="feature-header">
 <div class="feature-icon"> </div>
 </div>
 <h2 class="feature-title">PEP 701: f-string 语法形式化</h2>
 <div class="feature-subtitle">更灵活、更强大的 f-string 表达式</div>
</div>
</div>
<div class="feature-description">
 Python 3.12 正式采纳 PEP 701，解除了 f-string 表达式的许多限制。现在可以在 f-string 中使用反斜杠、多行表达式和相同引号嵌套。
</div>
```

```
<div class="code-container">
 <div class="code-header">
 <div class="code-title">示例代码：增强的 f-string 功能</div>
 <button class="run-btn" onclick="runCode(2)">运行代码</button>
 </div>
 <div class="code-block">
 <pre><code class="python"># 示例 1: f-string 中使用反斜杠（之前不允许）
items = ["apple", "banana", "cherry"]
print(f'列表项: {', '.join(items)}')

示例 2: 多行表达式和注释
x = 10
y = 20
result = f"""
总和: {x + y}
差值: {x - y}
乘积: {x * y}
商: {x / y if y != 0 else 'undefined'}
"""
print(result)

示例 3: 相同引号嵌套（之前需要转义）
name = "Alice"
现在可以在 f-string 中使用相同类型的引号
print(f'他说: "你好, {name}!"')

示例 4: 更复杂的表达式
def double(n):
 return n * 2

numbers = [1, 2, 3, 4, 5]
print(f'加倍后的数字: {[double(x) for x in numbers]}')</code></pre>
 </div>
 </div>

 <div class="output-container">
 <div class="output-title">输出结果：</div>
 <div id="output2">点击"运行代码"查看结果</div>
 </div>
</div>

<!-- 特性 3: PEP 709 - 内联行为 -->
<div class="feature-card">
 <div class="feature-header">
 <div class="feature-icon">⚡</div>
 <div>
 <h2 class="feature-title">PEP 709: 推导式内联</h2>
 <div class="feature-subtitle">推导式性能优化</div>
 </div>
 </div>
 <div class="feature-description">
 Python 3.12 通过 PEP 709 对推导式（列表、字典、集合推导式）进行了优化，使其在内部实现上更高效，减少了函数调用的开销。
 </div>
</div>
```

```
<div class="code-container">
 <div class="code-header">
 <div class="code-title">示例代码：推导式性能优化</div>
 <button class="run-btn" onclick="runCode(3)">运行代码</button>
 </div>
 <div class="code-block">
 <pre><code class="python">import time

示例 1: 列表推导式性能测试
print("列表推导式性能测试:")
start_time = time.time()
大型列表推导式
squares = [x**2 for x in range(1000000)]
list_time = time.time() - start_time
print(f"生成 100 万个平方数耗时: {list_time:.4f}秒")

示例 2: 字典推导式
print("\n字典推导式示例:")
words = ["apple", "banana", "cherry", "date", "elderberry"]
word_lengths = {word: len(word) for word in words}
print(f"单词长度字典: {word_lengths}")

示例 3: 嵌套推导式
print("\n嵌套推导式示例:")
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [num for row in matrix for num in row]
print(f"扁平化矩阵: {flattened}")

示例 4: 带条件的推导式
print("\n带条件的推导式示例:")
even_squares = [x**2 for x in range(20) if x % 2 == 0]
print(f"前 20 个偶数平方数: {even_squares}")</code></pre>
 </div>
 </div>

 <div class="output-container">
 <div class="output-title">输出结果: </div>
 <div id="output3">点击"运行代码"查看结果</div>
 </div>
</div>

<footer>
 <p>Python 3.12 新特性展示 | 代码可直接运行，无需额外修改</p>
 <p>注意：此演示使用 JavaScript 模拟 Python 运行环境，实际效果可能与真实 Python 3.12 环境略有差异</p>
</footer>
</div>

<script src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.8.0/highlight.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.8.0/languages/python.min.js"></script>
<script>
 // 初始化代码高亮
 hljs.highlightAll();
```

```
// 模拟 Python 运行环境
const pythonOutputs = {
 1: `Traceback (most recent call last):
File "<stdin>", line 5, in <module>
KeyError: 'nam'
Did you mean 'name'?

Traceback (most recent call last):
File "<stdin>", line 12, in <module>
AttributeError: 'Person' object has no attribute 'name'
Did you mean 'full_name'?

Traceback (most recent call last):
File "<stdin>", line 16, in <module>
ModuleNotFoundError: No module named 'matplotlib'
Did you mean 'matplotlib'?`,
```

```
2: `列表项: apple, banana, cherry
```

总和: 30  
差值: -10  
乘积: 200  
商: 0.5

他说: "你好, Alice!"  
加倍后的数字: [2, 4, 6, 8, 10],

```
3: `列表推导式性能测试:
生成 100 万个平方数耗时: 0.1253 秒
```

字典推导式示例:  
单词长度字典: {'apple': 5, 'banana': 6, 'cherry': 6, 'date': 4, 'elderberry': 10}

嵌套推导式示例:  
扁平化矩阵: [1, 2, 3, 4, 5, 6, 7, 8, 9]

带条件的推导式示例:  
前 20 个偶数平方数: [0, 4, 16, 36, 64, 100, 144, 196, 256, 324]`  
};

```
// 运行代码的函数
function runCode(featureNum) {
 const outputElement = document.getElementById(`output${featureNum}`);
 const runButton = event.target;

 // 显示加载状态
 outputElement.innerHTML = '执行中...';
 runButton.disabled = true;
 runButton.textContent = '执行中...';

 // 模拟执行延迟
 setTimeout(() => {
 outputElement.innerHTML = '${pythonOutputs[featureNum]}';
 runButton.disabled = false;
 runButton.textContent = '运行代码';
```



```
// 如果是特性 3，添加性能对比说明
if (featureNum === 3) {
 const note = document.createElement('div');
 note.style.marginTop = '10px';
 note.style.color = '#ffb74d';
 note.style.fontSize = '0.9rem';
 note.innerHTML = '注：Python 3.12 中推导式的执行速度比 3.11 快约 20-30%';
 outputElement.appendChild(note);
}
}, 800);
}

// 页面加载后自动运行第一个示例
window.addEventListener('DOMContentLoaded', () => {
 // 模拟自动运行第一个示例
 setTimeout(() => {
 document.getElementById('output1').innerHTML = `<span style="color:#81c784">${pythonOutputs[1]}</span>`;
 }, 1000);
});
</script>
</body>
</html>
...

```

## ## 功能说明

这个网页展示了 Python 3.12 的三个重要新特性：

1. **\*\*更友好的错误信息\*\*** - 改进了错误提示，提供更清晰的建议
2. **\*\*PEP 701: f-string 语法形式化\*\*** - 允许在 f-string 中使用反斜杠、多行表达式和相同引号嵌套
3. **\*\*PEP 709: 推导式内联\*\*** - 优化推导式性能，减少函数调用开销

## ## 使用说明

1. 将上面的完整代码保存为 HTML 文件（如`python312-features.html`）
2. 直接在浏览器中打开该文件
3. 点击每个特性卡片中的“运行代码”按钮查看示例输出
4. 页面已包含所有必要的样式和脚本，无需额外修改或安装

## ## 设计特点

- 响应式设计，适配各种屏幕尺寸
- 暗色主题，保护眼睛
- 交互式代码示例，可点击运行
- 详细的代码注释和解释
- 模拟 Python 3.12 运行环境

这个页面完全自包含，无需网络连接（除了初始加载的代码高亮库，但即使离线也能正常工作）。

> 性能指标：首字耗时：16.54 秒 | 总耗时：95.38 秒 | Token 数量：4552 | 端到端速度：47.73 tokens/s | 纯生成速度：57.74 tokens/s

---

## 11. 用户

求解以下数学问题，给出详细推导步骤：1. 计算定积分：  $\int_0^{\pi/2} (\sin x)^3 \cdot (\cos x)^2 dx$ ；2. 已知函数  $f(x) = x^3 - 6x^2 + 9x + 1$ ，求其单调区间和极值点。

---

## 12. AI

### 问题 1：计算定积分  $\int_0^{\pi/2} \sin^3 x \cos^2 x \, dx$

\*\*推导步骤：\*\*

1. 利用三角恒等式  $\sin^2 x = 1 - \cos^2 x$ ，将  $\sin^3 x$  改写为  $\sin x \cdot \sin^2 x = \sin x (1 - \cos^2 x)$ ，于是被积函数变为：

$$\begin{aligned} \int_0^{\pi/2} \sin^3 x \cos^2 x \, dx &= \int_0^{\pi/2} \sin x (1 - \cos^2 x) \cos^2 x \, dx \\ &= \int_0^{\pi/2} (\sin x \cos^2 x - \sin x \cos^4 x) \, dx \end{aligned}$$

2. 作变量代换  $u = \cos x$ ，则  $du = -\sin x \, dx$ ，即  $\sin x \, dx = -du$ 。积分上下限相应变化：当  $x = 0$  时， $u = \cos 0 = 1$ ；当  $x = \pi/2$  时， $u = \cos(\pi/2) = 0$ 。

3. 原积分转化为：

$$\int_0^{\pi/2} \sin x (\cos^2 x - \cos^4 x) \, dx = \int_1^0 (u^2 - u^4) (-du) = \int_0^1 (u^2 - u^4) \, du.$$

4. 计算定积分：

$$\int_0^1 (u^2 - u^4) \, du = \left[ \frac{u^3}{3} - \frac{u^5}{5} \right]_0^1 = \frac{1}{3} - \frac{1}{5} = \frac{5-3}{15} = \frac{2}{15}.$$

\*\*答案：\*\*  $\int_0^{\pi/2} \sin^3 x \cos^2 x \, dx = \frac{2}{15}$ 。

---

### 问题 2：已知函数  $f(x) = x^3 - 6x^2 + 9x + 1$ ，求其单调区间和极值点

\*\*推导步骤：\*\*

1. \*\*求导数\*\*：

$$\begin{aligned} f(x) &= x^3 - 6x^2 + 9x + 1 \\ f'(x) &= 3x^2 - 12x + 9 = 3(x^2 - 4x + 3) = 3(x-1)(x-3). \end{aligned}$$

2. \*\*求临界点\*\*：令  $f'(x) = 0$ ，解得  $x = 1$  和  $x = 3$ 。

3. \*\*分析单调性\*\*：用临界点将实数轴分成三个区间，考察各区间内  $f'(x)$  的符号。

- 当  $x < 1$  时，取  $x = 0$ ，则  $f'(0) = 9 > 0$ ，故  $f(x)$  在  $(-\infty, 1)$  上单调递增。
- 当  $1 < x < 3$  时，取  $x = 2$ ，则  $f'(2) = 3(1)(-1) = -3 < 0$ ，故  $f(x)$  在  $(1, 3)$  上单调递减。
- 当  $x > 3$  时，取  $x = 4$ ，则  $f'(4) = 3(3)(1) = 9 > 0$ ，故  $f(x)$  在  $(3, +\infty)$  上单调递增。

4. \*\*确定极值点\*\*：

- 在  $x = 1$  处， $f'(x)$  由正变负，故  $f(x)$  取得极大值。极大值为：

$$\begin{aligned} f(1) &= 1^3 - 6 \cdot 1^2 + 9 \cdot 1 + 1 = 5. \end{aligned}$$

- 在  $(x = 3)$  处,  $(f'(x))$  由负变正, 故  $(f(x))$  取得极小值。极小值为:

$$f(3) = 3^3 - 6 \cdot 3^2 + 9 \cdot 3 + 1 = 27 - 54 + 27 + 1 = 1.$$

**\*\*答案: \*\***

- 单调递增区间:  $(-\infty, 1)$  和  $(3, +\infty)$ ; 单调递减区间:  $(1, 3)$ 。

- 极大值点:  $(1, 5)$ ; 极小值点:  $(3, 1)$ 。

> 性能指标: 首字耗时: 21.80 秒 | 总耗时: 41.14 秒 | Token 数量: 1974 | 端到端速度: 47.98 tokens/s | 纯生成速度: 102.05 tokens/s

---

# 总体性能统计

**\*\*平均首字耗时\*\*:** 14.48 秒

**\*\*平均总耗时\*\*:** 54.76 秒

**\*\*平均 Token 数量\*\*:** 1783.5 tokens

**\*\*平均端到端速度\*\*:** 31.47 tokens/s

**\*\*平均纯生成速度\*\*:** 56.56 tokens/s